

A Dimensional Data Warehouse for Geospatial Monitoring of Municipal Public Works, with an Evolution Path Toward a Lakehouse Architecture

~~Anonymous Author(s)~~ [Uriel González Casiano](#)^[0009-0009-8172-9584], [Marco Tulio Maldonado Mejía](#)^[0009-0005-4691-9607], and [Gabriel Hurtado Avilés](#)^{*[0009-0002-5686-1822]}

~~Affiliation withheld for double-blind review~~ [Escuela Superior de Cómputo \(ESCOM\), Instituto Politécnico Nacional, Mexico City, Mexico](#)
{ugonzalezc2400, mmaldonadom2200}@alumno.ipn.mx
ghurtadoa@ipn.mx
gonzalezcasianouriel@gmail.com, marcotulioimm.s45@gmail.com,
gabrielhuav@gmail.com

Abstract. The accelerated growth of public infrastructure data in municipal governments generates heterogeneous volumes that are difficult to manage with isolated spreadsheets or transactional systems. This article presents the design and prototype implementation of a dimensional Data Warehouse for the geospatial monitoring of municipal public works, complemented by object storage (Cloudflare R2) for unstructured evidence —photographic reports and PDF documents— linked bidirectionally to the warehouse. The warehouse adopts a star schema with ten dimensions (five Slowly Changing Dimension Type 2, two Type 1, three Type 0) and two fact tables, and exposes OLAP queries and five analytical views for delay detection, budget alerts, and citizen participation through a single REST API. ~~A React/Leaflet geospatial map provides a~~, [released as reference code rather than as a hosted service. A published geospatial viewer offers](#) graph-like exploration ~~interface that links public across~~ works, communities, budgets, and citizen proposals. Rather than claiming a full enterprise lakehouse, we deliberately scope the contribution as a warehouse-centric design and outline its evolution toward a lakehouse (an open table format —Iceberg/Parquet— queried directly over the object store) as future work. The prototype was exercised over a seeded synthetic dataset of 1,247 public works from 55 communities and evaluated ~~for reproducibility through a published static frontend (Lighthouse) and a through a~~ documented, seeded protocol for the analytical detection module, [which recovers 84.1% of the injected anomalies with 99.5% precision](#). The design enables transparent, auditable, and citizen-oriented oversight of public works without the infrastructure costs of enterprise platforms.

Keywords: Data Warehouse · Dimensional Modeling · Slowly Changing Dimensions · Geospatial Monitoring · Open Government Data · Public Works · Object Storage · Lakehouse Architecture

* Corresponding author: gabrielhuav@gmail.com

1 Introduction

Transparency in the management of municipal public works is one of the most persistent challenges of digital governance in Mexico. Public-works departments (~~PWDs~~) simultaneously manage contractors, budgets, field supervisors, progress photographs, monthly reports, and hand-over processes; yet most municipalities operate with distributed file systems, spreadsheets, or, at best, isolated transactional systems that preclude historical analysis, early delay detection, and informed citizen participation.

The *lakehouse* paradigm [1] offers a reference response to this fragmentation: it unifies the flexibility of ~~the object storage typical of a Data Lake~~ Data Lake object storage (unstructured data, direct ingestion, low cost) with the analytical capability of a Data Warehouse (star schema, slowly changing dimensions, OLAP views). This work does not implement a full enterprise lakehouse: given the data volume at ~~the~~ municipal scale, we build a *dimensional Data Warehouse* as the analytical core and use object storage only as a repository of unstructured evidence linked to ~~the warehouse~~ it. Full lakehouse convergence —an open table format queried directly over the lake— is proposed as an evolution path (Section 8). ~~Applied to the municipal domain, this approach makes it possible to capture raw work progress photographs, record event audits, and compute physical and financial progress from a single platform.~~

This article presents ~~the design and implementation of~~ a prototype for the geospatial monitoring of public works in ~~the municipality of~~ Temascaltepec, State of Mexico ~~, with a population of~~ (64,844 inhabitants ~~distributed~~ across 55 communities~~).~~

Research Questions

RQ1. Can a dimensional Data Warehouse deployed on commodity managed services (PostgreSQL/Supabase plus S3-compatible object storage) provide auditable historical traceability of municipal public works at the scale of a rural Mexican municipality, without the enterprise platforms normally associated with lakehouse architectures?

RQ2. Which dimensional design decisions —fact grain, choice of Slowly Changing Dimension type per entity, and view materialisation strategy— are required so that delay detection, budget alerting and citizen participation can all be served from a single analytical schema?

RQ3. How effective is a threshold rule expressed over the warehouse’s analytical views at recovering anomalies in public-works execution, compared with an unsupervised baseline, and which anomaly classes does it fail to recover?

The three structure both the design and the evaluation: RQ1 is addressed by the architecture and warehouse design (Sections 3 and 4) and by the cost and deployment discussion (Section 8); RQ2 by the grain, SCD and view-strategy analysis (Section 4); and RQ3 by the evaluation of the detection module (Section 7.4).

Contributions

The main contribution is threefold: (1) a reference architecture centered on a dimensional *Data Warehouse* with slowly changing dimensions (SCD 0/1/2), complemented with S3-compatible object storage for unstructured evidence and exposed

through a public/private REST API (RQ1); (2) an interactive geospatial map that acts as a graph-like exploration interface, linking public works, communities, budgets, supervisors, construction companies, and citizen proposals (RQ2); and (3) a ~~social-impact methodological framework (evaluated on a test scenario and projected to real operation) that quantifies potential improvements in transparency, citizen participation, and operational efficiency~~ reproducible, non-circular evaluation of the analytical detection module against an independently injected ground truth, together with a failure analysis by anomaly class (RQ3).

Over a seeded test scenario, the prototype organizes 1,247 simulated public works with an accumulated budget of \$127.4 ~~million pesos, 8,934 simulated audit events, and 3,421 georeferenced photographic pieces of evidence.~~ As an illustration of the kind of analysis the warehouse enables, 34% of the works show a delay greater than 60 days, concentrated in communities with a lower socioeconomic development index (Spearman correlation $\rho = -0.67$, $p < 0.01$), a pattern that ~~if confirmed with real data would reveal territorial inequity in the execution of public infrastructure~~ million pesos (Section 7 details the full population). All figures reported in this article were obtained over synthetic data generated with a fixed seed; they characterise the behaviour of the system, not the municipality of Temascaltepec, and no claim about real public-works execution in that municipality is made or implied.

The remainder of the article is organized as follows: Section 2 reviews related work and contrasts the proposal with comparable systems; Section 3 describes the architecture; Section 4 details the warehouse; Section 5 describes the object storage layer; Section 6 presents the geospatial map; Section 7 reports results ~~and social impact~~; and Section 8 concludes.

2 Related Work

2.1 Data Lakehouse and Architecture Convergence

Zaharia et al. [1] coined the term *lakehouse* to describe an architecture that combines the open, flexible storage of a Data Lake with the ACID transaction guarantees, schema control, and analytical performance of a Data Warehouse. The transactional layer that makes this convergence possible was described by Armbrust et al. [11], who show how an open table format over cloud object stores can provide ACID semantics and time travel without a dedicated database engine; this is precisely the mechanism our object layer does not yet implement. Databricks [3] extended this model to the geospatial domain, ~~showing that~~ integrating vector and raster data on a unified platform ~~reduces query latency relative to decoupled systems. Recent work explores spatial data lakehouse architectures beyond purely vector or H3 formats [8] and positions the lakehouse as a reference model for unified data analytics [10].~~

In the present work we adopt the spirit of the lakehouse without requiring Apache Spark or Delta Lake: S3-compatible object storage (Cloudflare R2) acts as the ~~lake evidence~~ layer and PostgreSQL/Supabase as the warehouse layer, connected by a REST API ~~without an intermediate ETL through automated triggers. We note and by database triggers rather than by an external ETL process. We stress~~ that, in its current state, the object store ~~plays the role of is~~ a binary-evidence repository and not ~~of~~ a queryable analytical lake: no open table format (Iceberg/Delta/Hudi) ~~or and no~~ engine querying over the lake ~~is used yet are used~~. We therefore position the system as a dimensional Data

Warehouse with object storage, and reserve full lakehouse convergence ~~as a natural evolution of the design for future work.~~

2.2 Geospatial Knowledge Graphs

Knowledge graphs (KGs) have proven useful for integrating heterogeneous government data. Hogan et al. [4] provide a comprehensive survey of the state of the art in KGs, highlighting their application to georeferenced entities. Janowicz et al. [5] present KnowWhereGraph, one of the largest public geospatial graphs, which links human and environmental data under FAIR principles. In the open-government-data domain, Espinoza-Arias et al. [6] document the case of Zaragoza, where a municipal knowledge graph connects public services, infrastructure, and demographic data ~~to enhance transparency and citizen reuse.~~ Rajabi et al. [9] systematize the construction of knowledge graphs over open government data using Semantic Web technologies, ~~a framework that guides the relational design of our map.~~

Our geospatial map adopts an analogous ~~approach but~~ relational approach at a rural municipal scale: public works, communities, budgets, and citizen proposals constitute the entities ~~of the graph~~, while reports and photographs represent linked evidence. The distinction with respect to the works above is deliberate and is stated explicitly in Section 6: the graph is implicit in the foreign keys of the dimensional model and is materialised only at the interface level. Publishing the schema as an OWL vocabulary, which would place the system in the same family as [5,6,9], is the first item of our future work.

2.3 Open Government Data and Infrastructure Disclosure Standards

The literature on Open Government Data (OGD) [7] emphasizes that publishing public-infrastructure data in reusable formats increases accountability and facilitates the detection of irregularities. However, most OGD systems in Latin American municipalities are limited to static download portals, without analytical capability or integration with field data.

A directly relevant body of practice is the family of infrastructure disclosure standards. The Open Contracting for Infrastructure Data Standard (OC4IDS) [12], developed by the Open Contracting Partnership with CoST — the Infrastructure Transparency Initiative, defines a JSON schema for publishing project- and contract-level information across the infrastructure lifecycle. OC4IDS and our proposal are complementary: OC4IDS specifies *what* a government should disclose and in which interchange format, whereas we address *how* a small municipality can produce, historise and analyse that information internally. A natural convergence (Section 8) is to expose the warehouse as an OC4IDS-conformant endpoint, with *dim work* as the project entity and *fact audit events* as the source of lifecycle transitions.

This same approach of consolidating official open data into a queryable analytical artifact has been applied before in other government domains in Mexico, such as seismic risk [13][14], a methodological antecedent transferred here to the monitoring of public works. ~~The present system closes this gap by exposing the warehouse data directly on the public map, enabling parameterized queries and interactive geospatial visualization.~~

Table 1. Comparison with representative systems. SCD 2 = Slowly Changing Dimension Type 2; “Ev.” = managed bidirectional link between binary field evidence and the analytical store; “Sem.” = semantic layer.

System	Data model	History	Ev.	Geo UI	Sem.
KnowWhere Graph [5]	RDF knowledge graph	Snapshot	No	Yes	OWL/
RDF					
Zaragoza KG [6]	RDF knowledge graph	Partial	No	Partial	Ontology
Nova Scotia OGD KG [9]	RDF from OGD sets	No	No	No	Ontology
OC4IDS [12]	JSON disclosure schema	Lifecycle stages	By URL	Portal-dep.	JSON Schema
Geospatial lakehouse [3]	Delta/Parquet tables	Table versions	Raster/		
vector	Via BI	None			
Typical LatAm OGD portal	CSV/PDF downloads	No	No	Rare	None
This work	Dimensional schema	star	SCD 2 (5 dims)	Yes	Yes
					None (future work)

2.4 Positioning with Respect to Comparable Systems

Table 1 contrasts the proposal with representative systems along the dimensions that matter for municipal public-works oversight: the underlying data model, whether entity history is preserved, whether unstructured field evidence is a first-class citizen, whether a geospatial interface is provided, whether a formal semantic layer exists, and the deployment scale targeted.

Two gaps emerge. The semantic-web systems [5,6,9] provide a formal layer that we lack, but none historises entity state at audit granularity (“what did this record look like the day the budget was modified?”), which is what SCD 2 provides. And none links binary field evidence bidirectionally to the analytical store: in OGD portals and OC4IDS implementations, documents are referenced by URL without a managed lifecycle. This work fills the intersection of *temporal auditability* and *evidence linkage* at a scale where enterprise platforms are not economically viable.

3 System Architecture

Figure 1 illustrates the four ~~levels~~ layers of the proposed architecture, which follows a layered pattern with a clear separation of responsibilities ~~among ingestion, storage, processing, and presentation~~. It is described as a reference architecture and prototype: the analytical core is the dimensional Data Warehouse, and the object store operates as an evidence layer. The subsections below correspond one-to-one to the four layers labelled in the figure, so that the terminology of figure and text coincides.

3.1 ~~Ingestion~~ Layer 1: Ingestion

The actors (directors, supervisors, secretaries, ~~and planners~~ planners and registered citizens) interact with a web application built in HTML5/CSS3/JavaScript ~~and React/Vite modules for the map~~. Forms send structured data to the API (text, coordinates, amounts) and binary files (JPG/PNG, PDF) to the same endpoint, which routes them to their destination through an atomic transaction.

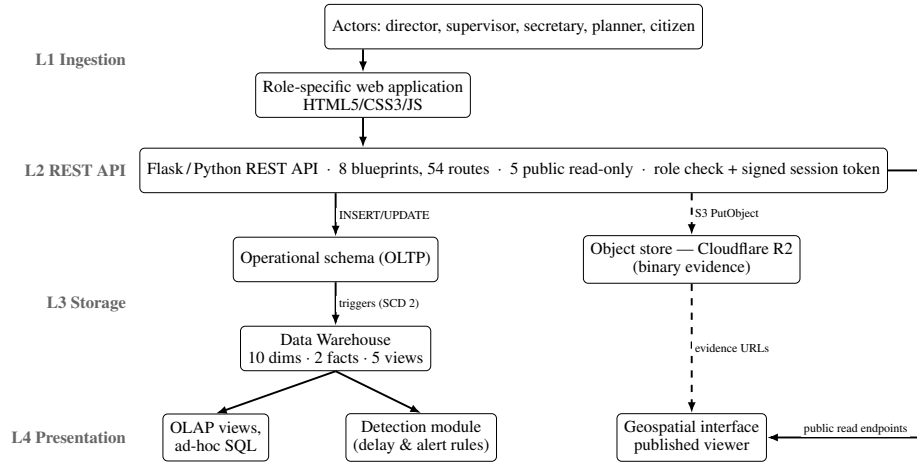


Fig. 1. System architecture (warehouse-centric): ~~the analytical core is the dimensional Data Warehouse and the object store (labeled “Data Lake”~~, redrawn in the figure) acts as the evidence layer^{English}. The four layers L1–L4 correspond to Sections 3.1–3.4. Solid arrows: structured-data flow; dashed arrows: ~~unstructured-data~~ unstructured (binary) evidence flow. Released role-specific pages read operational endpoints, while the warehouse serves detection and OLAP; the geospatial prototype is not a released artefact.

3.2 Object Storage Layer (Cloudflare R2): REST API

A Flask/Python REST API organised in eight blueprints exposes 54 routes, of which five are public and read-only (`/api/public/obras`, `/api/public/regiones`, `/api/public/resumen` and two evidence-listing routes). Access control for municipal staff is performed by a `@require_auth` decorator that validates a role identifier transmitted in request headers; the participatory-budget module for citizens uses HMAC-signed session tokens issued with `itsdangerous` over the application secret. Passwords are stored as PBKDF2-HMAC-SHA256 hashes with a 16-byte random salt. We describe the scheme precisely because it is deliberately lightweight and is *not* equivalent to a full token-based authentication service: hardening it (token expiry, rotation, and moving the staff role check from a header to a signed token) is listed among the limitations in Section 8.2.

3.3 Layer 3: Storage

Object store (Cloudflare R2). An S3-compatible object store that receives unstructured evidence through direct ingestion. Paths follow a *hive-partitioned* convention¹, shown in Equation (1):

$$\text{works}/\{\text{work_id}\}/\text{reports}/\{\text{year}\}-\{\text{month}\}/\{\text{ts}\}_{\{\text{slug}\}}.\{\text{ext}\} \quad (1)$$

This structure enables direct exploration by work and period; each object’s metadata (public URL, original name, MIME type, size in bytes, upload date) is also persisted in ~~the warehouse’s report_images~~ a warehouse-side table, creating a bidirectional link between the two layers.

3.4 Data Warehouse (PostgreSQL/Supabase)

Data Warehouse (PostgreSQL/Supabase). An analytical layer with a star schema, two fact tables, and ten dimensions (Section 4). The ~~analytical views computed over the fact tables feed the visualization layers directly~~ warehouse is populated from the operational schema by database triggers rather than by an external ETL job: eight triggers synchronise the dimensions (applying SCD 2 versioning where required) and four triggers emit rows into the audit fact table when a progress report, a cost record, an image upload or a citizen vote is registered. The warehouse uses range partitioning ~~on the fact_audit_events by year on the audit fact~~ table to optimize historical queries.

3.4 Presentation Layer 4: Presentation

~~A Flask/Python REST API exposes 22 endpoints (8 public without authentication, 14 private with header-based role control). The React~~ The released presentation layer consists of role-specific HTML/CSS/Leaflet map consumes the public endpoints and the warehouse views to render markers, technical cards, photo galleries linked to the object layer, and analysis layers. JavaScript pages that consume the REST API. Analytical consumption —the five views of Section 4.3, ad-hoc OLAP queries and the detection module of Section 3.5— is served directly from the warehouse. The geospatial viewer of Figure 3 is published alongside the rest of the artefact and reads a frozen projection of the same synthetic dataset, so map and workspaces never disagree.

3.5 Analytical Detection Module

The component evaluated in Section 7.4 is a stateless component that reads the warehouse and emits two artefacts: a *delay set*, from `v_delayed_works`, containing every work whose latest monthly fact row carries the delay flag together with its slippage; and an *alert stream*, from `v_audit_alerts`, which classifies audit events into five categories through a CASE expression over the fact table joined with the time, event-type and work dimensions.

The rule applied to a work-period record flags it as anomalous if *any* of three operational criteria holds:

- C1. *Cost outlier*: the executed budget deviates from the mean by more than three standard deviations, $|b_i - \mu_b|/\sigma_b > 3$.
- C2. *Extreme delay*: the recorded slippage exceeds 120 days.
- C3. *Progress inconsistency*: physical progress below 30% together with financial progress above 80%.

The corresponding anomaly score is $s_i = \max\left(\frac{|b_i - \mu_b|}{\sigma_b}, \frac{d_i}{120} \cdot \mathbb{I}[d_i > 120], 3.5 \cdot \mathbb{I}[C3]\right)$, where d_i is the slippage in days; the module flags $s_i \geq 3$. The score is used for the ranking-based metrics reported in Section 7.4. The alert stream is designed for presentation through the API.

Table 2. Warehouse dimensions and the type of SCD implemented.

Dimension	Modeled entity	SCD
dim_time	Calendar period	0
dim_work	Public work	2
dim_region	Community/neighborhood	2
dim_company	Construction company	2
dim_staff	Public-works department staff	2
dim_source	Funding source	0
dim_budget	Budget line	2
dim_citizen	Registered citizen	1
dim_proposal	Citizen proposal	1
dim_event_type	Catalog of 17 event types	0

3.6 A Note on Identifiers

This article uses English identifiers for tables, columns and views; the published implementation uses Spanish ones, as it was built with and for a Spanish-speaking municipal office. The correspondence is mechanical — `fact_audit_events` is `fact_eventos auditoria`, `v delayed works` is `v obras retraso`, `dim_work` is `dim obra`, and so on— and the repository README tabulates every pair, so that a reader inspecting the code can locate each object referenced here. This mapping is a prerequisite for the reproducibility claims of Section 7.

4 Data Warehouse Design

The warehouse adopts a dimensional star model [2] with two fact tables and ten dimensions (Table 2), shown schematically in Figure 2.

4.1 Fact Tables and Grain

We state the grain of each fact table explicitly, since it determines every subsequent aggregation and is a prerequisite for reproducing the design.

fact_audit_events: records audit `fact_audit_events` has a grain of *one row per audit event*: a single, atomic, immutable action performed on a work by an identified actor at an identified instant. It records events classified into 17 types (~~high amount, budget change, cancelled work, late event, among others~~ `work creation, work modification, state change, progress report, cost record, budget record, budget modification, image upload, permit issue, hand-over record, contractor selection, funding assignment, supervisor assignment, role change, proposal registration, vote cast and session start`) and is partitioned by year. ~~It contains~~ The seeded scenario writes 8,934 events (~~test data~~) ~~with~~ directly: the synchronisation triggers emit more as reports, costs, images and votes are registered, so a populated instance holds about 28,000. Each row carries foreign keys to the ten dimensions, financial metrics (~~budget amount, cost amount~~), progress metrics (~~physical progress pet, financial progress pet~~), and audit flags.

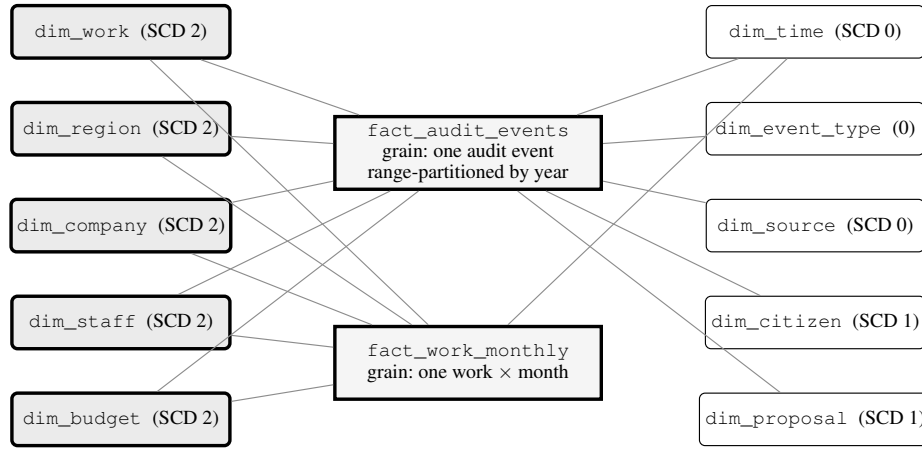


Fig. 2. Dimensional star schema of the Data Warehouse: two, redrawn for legibility. Two fact tables (*fact_audit_events*, *partitioned-range-partitioned* by year, and *fact_work_monthly*) are connected to the ten dimensions through surrogate keys. The five *slowly-changing dimensions* (SCD Slowly Changing Dimensions of Type 2 (shaded, left) are *highlighted to evidence* those that carry the historical traceability of the works. The monthly fact table is conformed only with the five SCD 2 dimensions and the time dimension.

fact_work_monthly aggregates *budget, cost, and physical progress by work and month* has a grain of *one row per work per calendar month*. It is a periodic snapshot table: for each work and month it stores the total budget, the accumulated cost, the remaining balance, the percentage executed, the physical progress, the number of reports and images registered, and a boolean delay flag with the associated slippage in days. Because the grain is a snapshot rather than a transaction, aggregating cost across months requires taking the last snapshot of the period rather than a sum; the analytical views encapsulate this rule so that consumers cannot get it wrong.

4.2 Slowly Changing Dimensions

The SCD 2 dimensions [2] (five in total: works, regions, companies, staff, and budget) maintain history through validity columns (*effective_date*, *expiration_date*, *is_current*), which makes it possible to reconstruct the exact state of any work at an arbitrary point in time.

Three representative use cases motivate this choice, none answerable with a Type 1 dimension. In a *budget-modification audit*, *essential for audits* *dim_budget* closes the current version and opens a new one, so an auditor can ask which amount was in force on the date a payment was authorised, not merely the amount in force today. Under *contractor reassignment*, *dim_company* preserves both versions, so progress recorded before the change stays attributed to the original contractor—a precondition for any performance analysis by contractor. Under *territorial reorganisation*, where communities are merged or renamed, *dim_region* versioning prevents a historical investment series from being silently rewritten.

The remaining dimensions need no versioning: `dim_time`, `dim_source` and `dim_event_type` are fixed catalogues (Type 0), while `dim_citizen` and `dim_proposal` are Type 1 because only their current state is analytically meaningful and retaining superseded personal records would be undesirable.

4.3 Analytical Views ~~Five materialized and Refresh Strategy~~

Five analytical views derived from the fact tables ~~feed the presentation layers~~ support analytical consumption: (1) ~~`v_delayed_works`~~ `v_delayed_works` (works with a schedule slip and days of delay); (2) ~~`v_audit_alerts`~~ `v_audit_alerts` (alerts by severity: ~~classified as~~ `HIGH_AMOUNT`, `CANCELLED_WORK`, `BUDGET_CHANGE`, `CANCELLED_WORKLATE_EVENT` and `REVIEW`); (3) ~~`v_progress_comparison`~~ `v_work_traceability` (physical vs. financial progress full SCD 2 version history of a work); (4) ~~`v_citizen_participation`~~ `v_citizen_participation` (votes and proposals by region); and (5) ~~`v_budget_executed`~~ `v_budget_executed` (accumulated budget execution by funding source).

These are standard SQL views rather than materialized ones. The choice is deliberate and determines the refresh strategy. At the volumes involved —of the order of 10^4 fact rows for 64,844 inhabitants— the views execute in tens of milliseconds over the partitioned fact table, so materialisation would buy little while introducing a staleness window; and since alerts must be actionable the moment a supervisor uploads a report, a live view is semantically preferable, with no refresh job to schedule or to fail. Should the deployment scale to several municipalities, the migration path is to materialise the aggregation-heavy `v_budget_executed` and `v_citizen_participation` with `REFRESH MATERIALIZED VIEW CONCURRENTLY` after each nightly load, keeping the two alert-bearing views live.

5 Object Storage Layer: Evidence Storage

The object layer uses Cloudflare R2, a service compatible with the Amazon S3 API, selected for its zero ~~read-operation egress~~ cost and reduced global latency. In its current state it functions as an object repository for binary evidence (images and PDFs) linked to the warehouse, and not as a queryable analytical lake: analytical queries are resolved in the Data Warehouse, not over R2. Adopting an open table format (~~Iceberg/Parquet~~) [11] that enables direct queries over the lake is proposed as future work (Section 8). The path design follows the *hive-partitioned* convention [3] of Equation 4(1).

Ingestion is direct from the web form: the supervisor attaches images or PDFs when creating a progress report; the API computes the object key, uploads it through the S3 SDK, records the metadata in the warehouse, and returns the ~~signed~~ public URL, all in a single atomic transaction. If the PostgreSQL insert fails, the upload to R2 is rolled back.

6 Geospatial ~~Map as a Knowledge-Graph~~ Interface ~~Prototype~~

The ~~map is the main interface of the system for citizens and authorities. It geospatial display~~ is an *implicit* graph at the interface level —the relationships are derived from the foreign keys of the dimensional model— and not a formal knowledge graph with

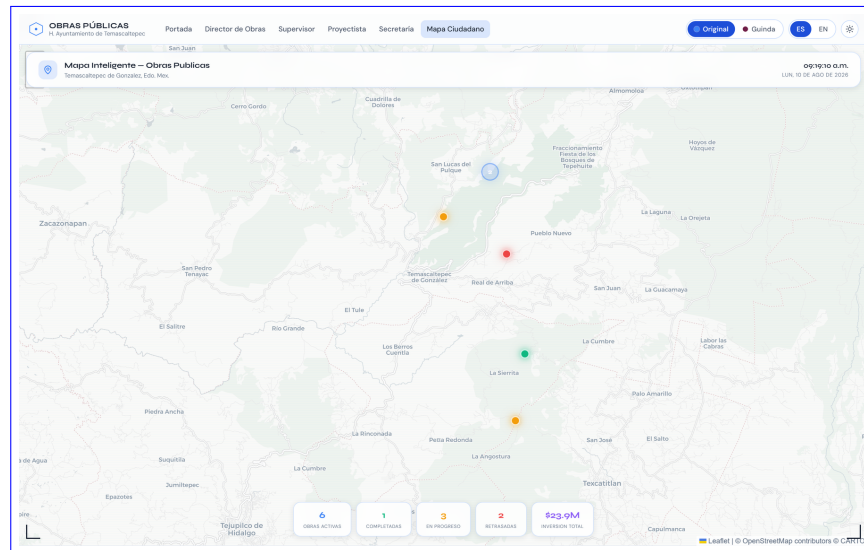


Fig. 3. Interactive geospatial map interface viewer, in light mode. Work markers are color-coded by state —green completed, audit-alert-panelamber in progress, and pop-up technical card with photographic evidence from red delayed— over the object layer. The instance shown is a demonstration environment with a reduced dataset (10 six works ; \$855K budget) that preserves the logical integrity of the complete dimensional model reported in demonstration dataset, with the abstract and in Section 7 indicator panel below. Live at https://gabrielhuav.github.io/PublicMunicipalWorks_DWH/mapa/?modo=claro, which reproduces this exact view.

an ontology or an RDF triple store; publishing the schema as a linked vocabulary is addressed in future work.

Figure 3 shows the main interface, with georeferenced markers color-coded by the state of the work (green, amber, red), the side panel of audit alerts, and the technical-card popup that displays photographic evidence and physical and financial progress indicators when a node is selected. It is part of the released artefact, reachable from the navigation bar of every page. Each marker sits at the real coordinates of its community, resolved from OpenStreetMap, with a deterministic offset of at most 150 m so that two works in the same community do not overlap. The community is therefore real; the exact site within it is not surveyed.

6.1 Management and Visualization Components

- **Marker layer:** each active work is represented by a marker color-coded by state —green (completed), amber (in progress), red (delayed)— derived from the `delayed_works` view, so the map reflects state changes as soon as the supervisor records a work's latest progress report.
- **Technical card (popup):** selecting a work displays its file number, name, community, total budget, assigned company, date range, a physical-progress bar, and an expandable section that loads on demand up to four for images from the object layer.

- **Analysis layer:** highlights delayed works ~~with red shading~~ and indicates the days of slippage (~~from v_delayed works~~ [criterion C2 of Section 3.5](#)); the dual physical/financial percentage detects works with high budget execution and low physical progress ([criterion C3](#)), a potential indicator of irregularities. ~~Alerts are presented in a side panel by type.~~

7 Results and Evaluation

7.1 ~~Performance Targets (Design)~~ [Scope of the Evaluation](#)

[Measured results, design targets and projections are kept strictly apart in what follows. We label every figure in this section as **measured** \(obtained by executing published code or auditing the published artefact, and reproducible by a third party: Sections 7.3 and 7.4\), **target** \(derived from architectural sizing and spot measurements of individual components, never from an end-to-end benchmark of a deployed system: the latency figures of Section 7.2\), or **projected** \(an expected operational outcome not yet observed in any deployment: Section 7.5\). No number in this article is a measurement taken on real municipal data.](#)

~~Table ?? reports the system’s design performance.~~

7.2 [Performance Targets \(Design, not Measured\)](#)

[The design targets for a reference configuration \(50 concurrent users, 95th percentile\) ~~, derived from the sizing of the architecture and from~~ are: 187 ms response time on /api/public/obras; 1.42 s initial map load; 543 ms for a four-image gallery; 127 req/s sustained; 89 ms to execute v_delayed works; 34 ms for an insert firing the SCD 2 trigger; and 112 ms read latency from the object layer. They derive from architectural sizing and spot measurements on individual components. ~~They are presented as design goals/projections and not as an integral production benchmark; , are targets in the sense of Section 7.1, and must not be read as a benchmark: none is a measurement of a deployed system, and the Flask+Supabase+R2 backend is distributed as a reference implementation \(Section 8.4\) and its whose permanent deployment is not guaranteed, so these figures should be taken as a projection until their reproducible end-to-end verification.~~ The test population reproduces the intended operating structure: 1,247 works across 55 communities, 8,934 \[seeded\]\(#\) audit events, 3,421 photographic pieces of evidence, \\$127.4~~million~~ \[million pesos\]\(#\) in budget, 2,156 citizen proposals, and 8,723 votes cast.](#)

~~Design performance targets (reference configuration, 50 concurrent users, p95).~~

7.3 ~~Benchmark Protocol~~ [Evaluation of the Published Frontend \(Reproducible Measured\)](#)

~~The performance-evaluation protocol is documented for its reproducibility end-to-end once the backend is deployed. Test client: Dell Latitude 7420 (Intel Core i7-1185G7, 32 GB DDR4-3200, Fedora 44). The static artefact is permanently online, so its delivery can be audited by anyone against a public URL. Table 3 reports Google Lighthouse over its two entry points, the landing page and the vendored viewer. These are measurements in the sense of Section 7.1, not targets.~~

Table 3. Lighthouse audit of the published artefact, median of three runs per page. Reproducible over the public URL.

Page	Core Web Vitals					Scores (0–100)			
	FCP	LCP	SI	TBT	CLS	Perf.	A11y	B.P.	SEO
Landing	3.2 s	3.3 s	4.0 s	0 ms	0.000	85	100	96	90
Map	3.6 s	5.3 s	3.7 s	0 ms	0.000	72	100	93	100

Both pages score 100 –Mbps symmetric link, ~25 ms latency to the server). Tools: Locust 2.20.0 (50 concurrent users, spawn rate 2/s, 3 min per run), PostgreSQL EXPLAIN (ANALYZE, BUFFERS) (10 iterations per view), psql \timing, and eURL (time_starttransfer and time_total). We report the p95 of: API response time (end-to-end), SQL view execution, insert with SCD 2 trigger, and signed-URL resolution latency in R2. Conditions: database with ~on accessibility, leave the main thread idle and shift no layout, so only delivery weight separates them: the landing page waits on render-blocking web fonts and a CDN animation library, the map on a 533 kB bundle. Neither is a property of the warehouse: both are traits of an unoptimised front end, remediable without touching the architecture.

Conditions: Lighthouse 12.8.2, headless Chrome 151, emulated mobile with simulated throttling, 11 ,000 records, PostgreSQL cache invalidated before each test, no concurrent loads, 3 independent runs per metric (median value) August 2026, tag v1.4.0-icokg2026; the artefact is static, so a re-run audits the same page. Lighthouse measures delivery, not query latency.

7.4 Evaluation of the Delay/Alert Detection Module (Measured)

Ground truth. Deriving the ground truth from the same predicate used by the detector would make precision equal to 1 by construction and the comparison uninformative. We therefore take the ground truth from an independent source: the data generator injects anomalies, with probability 0.15 and in four classes —*cost overrun* (the executed budget is multiplied by a factor drawn from [1.5, 3.0]), *delay* (slippage drawn from [120, 365] days), *inconsistency* (physical progress forced to [5, 25]%) and *ghost work* (physical progress forced to 0)— and records the class in a label field that the detector never observes. That label is our ground truth. Over the seeded dataset, 214 of 1,421 work-period records (15.06%) are anomalous.

7.5 Evaluation of the Published Frontend (Lighthouse)

Independently of the backend,

Setting. The dataset comprises 1,421 work-period records across the 55 communities of Temascaltepec, generated with seed 42. It is a distinct artefact from the 1,247-work warehouse dataset used elsewhere in this section, and the two must not be conflated. The detector is the rule of Section 3.5. The baseline is an Isolation Forest (100 estimators, contamination 0.15, seed 42) over five normalised features: executed budget, physical progress, financial progress, slippage and the published frontend can be evaluated reproducibly with Google Lighthouse. Table 3 reports the *Core Web Vitals* metrics over the published site; being a browser audit over a public URL, any reviewer can reproduce it. community development index.

~~Metries of the published frontend (Lighthouse). Reproducible over the public URL.~~

~~Table 4. Detection performance against an independently injected ground truth (synthetic dataset, 1,421 records, seed 42). Reproducible with the published script.~~

Method	Precision	Recall	F1	ROC-AUC	PR-AUC
Threshold rule (this work)	0.9945	0.8411	0.9114	0.9356	0.8702
Isolation Forest (baseline)	0.8545	0.8505	0.8525	0.9855	0.9289

~~Audit run with Lighthouse v13.2.0 (Chrome 149, emulated Moto G Power device, simulated 4G network with throttling) over the repository’s public URL (last run: 2026-06-22). Lighthouse evaluates page performance, not query latency; the latter corresponds to Table ?? and requires the deployed backend.~~

Explanation of the results. Table 4 shows a clear division of labour between the two methods, and the interesting content of the experiment is in that division rather than in the headline F1.

7.5 Evaluation of the Delay/Alert Detection Module

The analytical component—detection of delayed works and classification of audit alerts through the `v_delayed_works` and `v_audit_alerts` views—is evaluated as a classifier. Since the source has no real labels of “problematic delay”, a controlled and reproducible protocol is defined: over the test set, every record that meets the operational criterion (e.g., a slip > 60 days, or financial progress exceeding physical progress by a given threshold) is labeled positive. The threshold rule is almost perfectly precise (0.9945: a single false positive in 1,421 records) but recovers only 84.1% of the anomalies. Its 34 misses are not distributed uniformly across classes. Decomposing recall by injected class, the rule recovers 65/65 ghost works, 57/57 inconsistencies and 43/43 delays—all at recall 1.0—but only 15/49 cost overruns (0.3061). The reason is structural: criterion C1 flags a budget only beyond three standard deviations, whereas the injected overrun multiplies the budget by a factor of at most 3 over a distribution that is already right-skewed across heterogeneous work types. A moderate overrun on an already-expensive work therefore remains inside the 3σ envelope. This is a genuine and actionable design finding: a global σ threshold is the wrong instrument for cost anomalies, and the rule output is contrasted against that ground truth. Table 4 reports precision, recall, F1, and the areas under the ROC and precision–recall curves, together with an unsupervised baseline detector (Isolation Forest), over the synthetic dataset of $\sim 1,247$ public works in Temascaltepec generated with a fixed seed (42). criterion should be computed *per work type* rather than over the whole population.

Evaluation of the detection module against a ground truth defined by an operational criterion (synthetic dataset; seed 42). Reproducible with the published script. The Isolation Forest inverts the trade-off. Its recall is marginally higher (0.8505) but it produces 31 false positives against the rule’s single one, and its precision drops to 0.8545. Its ranking quality is nevertheless clearly superior (ROC-AUC 0.9855 versus 0.9356; PR-AUC 0.9289 versus 0.8702), because it exploits the joint distribution of the five features rather than three independent axis-aligned cuts. For an oversight application the operating point matters more than the ranking: an alert panel that a municipal officer reviews manually is dominated by precision, since false alarms erode

trust in the tool. The rule is therefore the better default, and the baseline indicates how much headroom a learned detector would offer if recall on cost overruns were the priority.

7.5 Illustrative Analytical Capability (Projected Social Impact)

Methodological note. The results in this subsection were obtained over a synthetic dataset generated with Python's Faker library (seed 42), which replicates the dimensional structure and the volume the system would process in a real deployment in Temascaltepec; ~~they do not constitute empirical~~. They demonstrate that the analytical pipeline runs end to end and produces the intended artefacts; they are not evidence about the municipality, and the patterns described below are properties of the generator, not findings.

Analytical capability. ~~To demonstrate the detection of territorial inequity patterns, a controlled synthetic correlation was generated~~ What the architecture can compute. A controlled correlation was injected between the simulated ~~socioeconomic development index (SDI)~~ socio-economic development index and the days of delay ($\rho = -0.67$, ~~and the pipeline recovers it~~ ($\rho = -0.67$, $p < 0.01$, $n = 55$ communities). ~~The correlation is not real evidence of inequity, but shows that the architecture can~~ What this demonstrates is not the coefficient, which was placed there by construction, but that the system can compute Spearman correlations over dimensional data, identify territorial delay patterns, generate alerts based on statistical analysis, and visualize join them to the region dimension, raise alerts and render them geospatially.

Over the same synthetic set, two patterns are illustrated: (i) **budget concentration**, with 60% of the budget (\$76.4M) in 20% of the works (249 high-impact works) and the remaining 80% (998 projects) sharing 40% (\$51.0M), a Pareto-like distribution that suggests prioritizing large projects over community maintenance; and (ii) **differentiated participation**, with an average of 4.2 votes per proposal (high variance, $\sigma = 3.8$) and 2.3 times more votes in communities with internet access above 70%, indicative of a digital divide. A Pareto-style budget concentration analysis and a comparison of participation rates by connectivity were exercised in the same way. Whether such patterns exist in Temascaltepec is an empirical question this article does not answer.

As design goals and projections Projected operational benefits. ~~over a six-month operating horizon, the system anticipates: reducing the monthly report generation time to ~4.2 h (vs. 2–3 manual days) and~~ We state these qualitatively rather than numerically, since any point estimate would require a derivation that the present evidence does not support. The system is designed to reduce the time needed to assemble a monthly progress report, to lower the volume of formal information requests by ~67% (access to a work's information in ~2.3 s vs. 3–5 business days); and enabling accountability control through automatic audit alerts, detection of budget anomalies, support to investigations, and the identification of irregularities equivalent to a potential recovery of \$2.4M pesos through continuous monitoring. These figures are design goals, pending validation in real operation, making the same data publicly queryable, and to surface budget anomalies early enough to act on them. Quantifying any of these requires a pre/post measurement in the municipal office, the first item of the validation plan below.

8 Conclusions and Future Work

8.1 Conclusions

This work presented a pragmatic architecture centered on a dimensional Data Warehouse for the geospatial monitoring of municipal public works, ~~which complements~~ complementing the analytical store with S3-compatible object storage for unstructured evidence under a single REST API. ~~The proposal shows that it is possible to build a transparency system with analytical capability (OLAP views, SCD 2 history), georeferenced photographic evidence, and citizen participation, without relying on high-cost enterprise platforms such as Apache Spark or Databricks. Full lakehouse convergence is identified as the natural evolution of the design and is detailed in future work.~~

~~The main contributions were: (1) **Reference architecture:** a dimensional Data Warehouse (PostgreSQL/Supabase) with five~~ Returning to the research questions: RQ1 is answered affirmatively at the design level —ten dimensions, five of them SCD 2 dimensions, complemented with object storage (Cloudflare R2) and a Flask REST API, with an estimated operating cost of , two fact tables and five views on managed services at an estimated \$47 USD/month for a municipality of 64,844 inhabitants; (inhabitants— with the caveat that no measurement on a stable production deployment exists yet. RQ2 is answered in Section 4: an event-grain transaction fact table plus a work-month snapshot table, with SCD 2) **Graph-like geospatial interface:** ~~the React/Leaflet map unifies structured warehouse data (progress, alerts, budget) with unstructured evidence (photographs, documents); (3) **Illustrative analysis:** over the test set, the identification of territorial inequity ($\rho = -0.67$), budget concentration (60%/20%), and a digital divide is exemplified, pending validation with real data; and (4) **Projected social impact:** goals of reducing formal information requests, converting citizen proposals into works, early delay detection, and recovery of resources~~ restricted to the five audit-relevant entities, suffices to serve delay detection, budget alerting and participation analytics from one schema. RQ3 is answered in Section 7.4: the rule attains 0.9945 precision at 0.8411 recall against an independent ground truth, with failures concentrated in a single anomaly class whose criterion is mis-specified.

The proposal shows that it is possible to build a transparency system with analytical capability (OLAP views, SCD 2 history), georeferenced photographic evidence, and citizen participation, without relying on high-cost enterprise platforms.

8.2 Limitations

~~(1) The dynamic deployment uses Render's free tier, whose PostgreSQL database has limited validity and whose web service is suspended on inactivity; permanence is delegated to the static version (which operates over the data included in the repository and requires no server), with a planned migration to a provider that does not suspend the service on inactivity. There is no hosted dynamic deployment: permanence rests on the static artefact, and the API is distributed to be run by whoever needs it rather than kept online by us. (2) The participation module uses a simple voting scheme that may not adequately represent minorities; identity validation via the national ID (INE credential) is planned. (3) The quality of the analysis depends on the quality, coverage, and freshness of the data entered by field supervisors. (4) The analytical results (including $\rho = -0.67$) come from a synthetic dataset.~~ All analytical results come

from synthetic data (Faker, seed 42), due to privacy constraints on the open publication of municipal data; no result in this article has been validated against real municipal records. (5) The architecture is, in its current state, *data-warehouse-centric*: the object store holds binary evidence but is not queried analytically; ~~full lakehouse convergence is not implemented and is proposed as future work.~~ (6) The public map endpoints read the operational schema rather than the warehouse views, so the two read paths can diverge; unifying them is pending. (7) Access control is lightweight—the staff role travels in a request header and the citizen session token, though HMAC-signed, has no expiry—which is adequate for a prototype whose analytical surface is public by design, but must be hardened before handling non-public data.

8.3 Future Work

(0) Evolve the object layer toward a full lakehouse by adopting an open table format [11] (Apache Iceberg, or Parquet with transactional metadata) queryable directly over R2 through a lightweight engine (such as DuckDB or Trino), without introducing Apache Spark, so that the “lake” shifts from being only evidence to being analytically queryable; (1) a computer-vision inference pipeline (YOLOv8, SAM) to automatically classify work progress from the stored photographs; (2) publishing the schema as an OWL vocabulary to interoperate with Linked Data, and exposing an OC4IDS-conformant [12] projection of the warehouse so that the municipality’s data becomes comparable with other CoST-aligned jurisdictions; (3) quadratic voting to improve the representativeness of minority proposals; (4) time-series models (Prophet, LSTM) to predict completion dates; and (5) recomputing the cost-anomaly criterion per work type, following the failure analysis of Section 7.4, and re-evaluating against the same ground truth; and (6) validating the architecture in 3–5 additional municipalities of the State of Mexico, with a pre/post measurement of report-preparation time and information-request volume.

8.4 Availability

The source code is distributed in two anonymized public repositories. The **backend** (Flask with JWT authentication, The warehouse DDL, ETL scripts, SCD 2 triggers, and a synthetic-data generator with seed 42) is available at ; the **React/Leaflet frontend** at . The guaranteed-availability artifact is seeded generators, load-testing scripts and the **static demo** (), which runs entirely on the client over the data included in the repository and requires no server. Additionally, a best-effort **dynamic deployment** is offered (Flask+Supabase+R2 backend on Render’s free tier), which connects in real time to evaluation script are archived at https://github.com/gabrielhuav/PublicMunicipalWorks_DWH. A static demonstration is published through GitHub Pages at https://gabrielhuav.github.io/PublicMunicipalWorks_DWH/. It preserves the interface of the original public-works prototype—its four role workspaces, the citizen-participation module and the map—replacing remote authentication and writes with synthetic in-browser behaviour, and adds Spanish/English switching and two colour themes in light and dark mode. It therefore makes no request to any backend service. Access is open by construction: the credentials are printed on the landing page and the form accepts any value, even none; every screen is served from a synthetic dataset held in the visitor’s own tab, so no input can affect another visitor.

The Flask implementation in `backend/` remains the operational reference, where the same DEMO- accounts are constrained to read-only access. It is runnable, not archived: `docker/` raises PostgreSQL and the object layer but **does not guarantee permanent availability** (it is suspended after inactivity). Analytical access to the map is public and open in both modalities; JWT authentication (bcrypt password hashing, configurable expiration) is reserved for administration functions, and demonstration credentials are documented in the README API with the warehouse and its seeded population, so an agency wanting to operate the system starts from a working stack; the figures above were verified against that container. Table 4 is reproduced by running `generar_dataset_obras.py` and `eval_deteccion_v2.py` from `scripts/evaluacion/`, both with seed 42. The population figures are constants in `scripts/generate_synthetic_data.py`, met by construction; its `--verificar` flag checks them without a database.

Declaration on the Use of Generative AI

The authors used a generative AI assistant, under their direction, for language editing, L^AT_EX formatting, drafting text subsequently revised and approved by the authors, and writing the auxiliary scripts distributed with the artefact, and generating the artefact's illustrative work images, each labelled as such in the file. The research questions, the dimensional design, the ETL, the data collection, the experiments and the interpretation of the results are the authors' own work.

Disclosure of Interests

The authors declare that they have no conflicts of interest relevant to the content of this article.

References

1. Zaharia, M., Chen, A., Davidson, A., Ghodsi, A., Hong, S.A., Konwinski, A., Murching, S., Nykodym, T., Ogilvie, P., Parkhe, M., Singh, F., Xin, R. et al.: Lakehouse: A New Generation of Open Platforms that Unify Data Warehousing and Advanced Analytics. *a new generation of open platforms that unify data warehousing and advanced analytics*. In: *Proc. Proceedings of the 11th Conf. Conference on Innovative Data Systems Research (CIDR 2021)*. Online (2021)
2. Kimball, R., Ross, M.: The Data Warehouse Toolkit: The Definitive Guide to Dimensional Modeling, 3rd edn. Wiley, Indianapolis (2013). ISBN-978-1-118-53080-1
3. Palkar, S., Thusoo, A., Boncz, P.A.: Building a Geospatial Lakehouse, Part-geospatial lakehouse, part 1. Databricks Engineering Blog (2021). <https://www.databricks.com/blog/2021/12/17/building-a-geospatial-lakehouse-part-1.html>, last accessed 2026/01/. Last accessed 15 online resource Jan 2026
4. Hogan, A., Blomqvist, E., Cochez, M., d'Amato, C., de Melo, G., Gutierrez, C., Kirrane, S., Labra Gayo, J.E., Navigli, R., Neumaier, S., Ngonga Ngomo, A.C., Polleres, A., Rashid, S.M., Rula, A., Schmelzeisen, L., Sequeda, J., Staab, S., Zimmermann, A. et al.: Knowledge Graphs. *ACM Computing Surveys graphs. ACM Comput. Surv.* **54**(4), Article 71 (2021). <https://doi.org/10.1145/3447772>

5. Janowicz, K., ~~Hitzler, P., Li, W., Rehberger, D., Schildhauer, M., Zhu, R., Shimizu, C., Fisher, C.K., Cai, L., Mai, G., et al.~~: Know, ~~Know-Where~~[know where](#), KnowWhere-Graph: ~~A-Densely-Connected, Cross-Domain Knowledge Graph and Geo-Enrichment Service Stack for Applications in Environmental Intelligence.~~ [AI Magazine a densely connected, cross-domain knowledge graph and geo-enrichment service stack for applications in environmental intelligence.](#) *AI Mag.* **43**(1), 30–39 (2022). <https://doi.org/10.1002/aaai.12043>
6. Espinoza-Arias, P., ~~Fernández-Ruiz~~[Fernández-Ruiz](#), M.J., ~~Morlán-Plo~~[Morlán-Plo](#), V., Notivol-Bezares, R., Corcho, O.: The Zaragoza's ~~Knowledge Graph: Open Data to Harness the City Knowledge~~[knowledge graph: open data to harness the city knowledge](#). *Information* **11**(3), 129 (2020). <https://doi.org/10.3390/info11030129>
7. ~~Lourenço~~[Lourenço](#), R.P.: An analysis of open government portals: ~~A-a perspective of transparency for accountability.~~ [Government Information Quarterly Gov. Inf. Q.](#) **32**(3), 323–332 (2015). <https://doi.org/10.1016/j.giq.2015.05.006>
8. ~~Forrest, M.: Spatial Data Lakehouse Architecture: Beyond H3 and Vector-Only Data.~~ [forrest.nyc \(2025\).](#), last accessed 2026/01/15 online resource
9. Rajabi, E., Midha, R., de Souza, J.F.: Constructing a ~~Knowledge Graph for Open Government Data: The Case~~[knowledge graph for open government data: the case](#) of Nova Scotia ~~Disease Datasets.~~ [Journal of Biomedical Semantics disease datasets.](#) *J. Biomed. Semant.* **14**, 4 (2023). <https://doi.org/10.1186/s13326-023-00284-w>
10. ~~Oracle Corporation: What is a Data Lakehouse? Oracle Big Data Documentation (2025).~~, last accessed-
11. ~~Armbrust, M., et al.~~: Delta Lake: high-performance ACID table storage over cloud object stores. *Proc. VLDB Endow.* **13**(12), 3411–3424 (2020). <https://doi.org/10.14778/3415478.3415560>
12. ~~Open Contracting Partnership, CoST — the Infrastructure Transparency Initiative: Open Contracting for Infrastructure Data Standard (OC4IDS).~~ <https://standard.open-contracting.org/infrastructure/latest/en/>. Last accessed 1 Aug 2026 ~~/01/15 online resource~~
13. ~~Anonymous Author(s): Reference by the authors, withheld for double-blind review-~~
14. ~~Villa Vargas, J.M., Hurtado Avilés, G., Climent Hernández, J.A.:~~ Cuando México tiembla: la historia contada por los datos. *AZCATL Rev. Divulg. Cienc. Ing. Innov.* **4**(6), 28–33 (2026). <https://doi.org/10.24275/AZC2026E1004>